

PERSONAL REFERENCE

JavaScript Classes & Inheritance

A from-scratch guide, with C# parallels throughout

If you remember nothing else, remember this:

an instance is always ONE object. Inheritance does not stack a parent object under a child object. There is one object, and the constructors — parent first, then child — take turns writing onto it.

`super()` is just "now run the parent's constructor against *this same object*."

What's inside

- 1 The mental model — objects and the prototype link
- 2 The old school — building a "class" with a function
- 3 The old school — inheritance by hand (this is where it clicks)
- 4 ES6 `class` — the same thing with nicer syntax
- 5 `super()` and the derived constructor — the part you keep missing
- 6 JavaScript vs C#, side by side
- 7 `this`, detachment, and `bind` (the related trap)
- 8 Cheat sheet + your specific mistakes, corrected

PART 1

The mental model: objects and the prototype link

Everything about JS "classes" is built on one small idea. Get this idea and the rest is bookkeeping.

An object can point at another object

Every JS object has a hidden internal link to *another* object. That link is called **[[Prototype]]** (the double brackets just mean "internal, not a normal property you typed"). When you read a property off an object, JS does this:

1. Does the object have this property *directly* (its own)? Use it.
2. If not, follow the **[[Prototype]]** link to the next object and look there.
3. Keep walking up the chain until found, or until the link is **null** (then the value is **undefined**, or a method call throws).

That walk up the chain is inheritance in JavaScript. There is no magic class machinery underneath — just objects linked to objects.

```
const animal = { legs: 4 };
const dog = { bark: true };
Object.setPrototypeOf(dog, animal); // dog's [[Prototype]] now points at animal

console.log(dog.bark); // true - own property
console.log(dog.legs); // 4 - not on dog, found on animal via the link
console.log(dog.wings); // undefined - walked the chain, hit null, nothing found
```

KEY IDEA

Inheritance = "if I don't have it, look at the object I'm linked to." A chain of objects. Nothing is copied down; the links are followed live at read time.

The naming trap: **.prototype** is NOT **[[Prototype]]**

This confuses everyone, so pin it down once:

Term	What it is	Lives on
[[Prototype]] (a.k.a. __proto__)	The live link from an object <i>up</i> to the object it inherits from.	Every object (instances included).
.prototype	A plain property on a <i>constructor function / class</i> . It holds the object that will become the [[Prototype]] of instances that function creates.	Functions and classes only — not on instances.

So `new Dog()` makes an object whose `[[Prototype]]` is set to `Dog.prototype`. One is the link on the instance; the other is the template object hanging off the constructor. Same word, two different things.

C# CONTRAST

In C#, a type's method table is fixed at compile time and baked into the type. In JS, "the methods" live on a *regular object* (`Dog.prototype`) that you can inspect and even mutate at runtime. The chain is data, not a sealed type definition.

PART 2

The old school: building a "class" with a function

Before `class` existed (pre-2015), you made "classes" with plain functions. Learning this first is the fastest way to truly understand `class`, because `class` is literally this with sugar on top.

A constructor function

Any function called with `new` becomes a constructor. By convention it's capitalized.

```
function Animal(name) {  
  // inside here, `this` is the brand-new object being built  
  this.name = name;  
}  
  
// Shared methods go on the .prototype object — ONE copy, shared by all instances  
Animal.prototype.describe = function () {  
  return `${this.name} the animal`;  
};  
  
const a = new Animal("Rex");  
console.log(a.name);           // "Rex"      — own property, set in the constructor  
console.log(a.describe());    // "Rex the animal" — found on Animal.prototype via the chain
```

What `new` actually does (memorize these 4 steps)

When you write `new Animal("Rex")`, JS does exactly this:

1. Create a fresh empty object: `{}`.
2. Set that object's `[[Prototype]]` to `Animal.prototype`.
3. Call `Animal` with `this` pointing at the new object (so `this.name = name` writes onto it).
4. Return that object automatically (unless the constructor explicitly returns a different object).

KEY IDEA

The constructor doesn't *create* the object — `new` does, in step 1. The constructor just *fills it in*. Hold onto that: it's the whole resolution to the `super()` puzzle later.

Notice where things live: **data** (`name`) ends up *on the instance* (set via `this.`), while **methods** (`describe`) live *once* on `Animal.prototype` and are shared by every instance through the link. That's why 10,000 animals don't each carry their own copy of `describe`.

PART 3

The old school: inheritance by hand

This is the section to reread whenever `super()` feels slippery. Doing inheritance manually makes the "one object" truth impossible to miss.

Say `Dog` should inherit from `Animal`. Three things have to happen, and each maps directly onto something `class` does for you automatically:

```
function Dog(name, breed) {
  Animal.call(this, name); // (1) run the parent constructor ON THIS SAME object
  this.breed = breed;      // then add the child's own stuff
}

// (2) link the chains so Dog instances can find Animal's methods
Dog.prototype = Object.create(Animal.prototype);

// (3) repair the constructor pointer (Object.create wiped it)
Dog.prototype.constructor = Dog;

// child's own shared method
Dog.prototype.bark = function () { return "woof"; };

const d = new Dog("Mitzie", "Corgi");
console.log(d.name);      // "Mitzie" - written by Animal's ctor, onto the one object
console.log(d.breed);      // "Corgi" - written by Dog's ctor, onto the same object
console.log(d.describe()); // "Mitzie the animal" - Animal.prototype, up the chain
console.log(d.bark());     // "woof" - Dog.prototype
```

Look closely at line (1) — this is `super()`

`Animal.call(this, name)` means: "call the `Animal` constructor, but use MY `this` as its `this`." The `Animal` constructor runs its `this.name = name` — and "this" is the exact same object `Dog` is building. There is no second object. `Animal` isn't making its own instance; it's just running its setup code against the one object that already exists.

THIS IS THE WHOLE THING

`super(name)` in a modern class is **exactly** `Animal.call(this, name)`: "run the parent constructor against this same object." When you next feel tempted to say "super() creates the parent instance and the child layers on top" — no. Picture `Animal.call(this)`. One object. The parent constructor just writes onto it.

The three jobs, and who does them in modern `class`

Manual (old school)	What it achieves	Modern <code>class</code> equivalent
<code>Animal.call(this, name)</code>	Run parent's constructor on the one object.	<code>super(name)</code>
<code>Dog.prototype = Object.create(Animal.prototype)</code>	Wire instance → Dog.prototype → Animal.prototype so method lookup walks all the way up.	<code>class Dog extends Animal</code>
<code>Dog.prototype.constructor = Dog</code>	Keep <code>d.constructor</code> pointing at Dog.	Handled automatically by <code>extends</code> .

The resulting chain (for our `d`)

```
d —[[Prototype]]→ Dog.prototype —[[Prototype]]→ Animal.prototype —[[Prototype]]→ Object.prototype → null

own on d:          bark()                describe()
toString(), etc.
name, breed
```

Reading `d.describe()` walks: not on `d` → not on `Dog.prototype` → found on `Animal.prototype`. That walk is the entire inheritance mechanism.

PART 4

ES6 `class`: the same thing, nicer syntax

`class` arrived in 2015. It does not add a new object model. It's **syntactic sugar** over exactly Part 3. Same prototype chain, same one object, less typing.

ES6 CLASS

```
class Animal {
  constructor(name) {
    this.name = name;
  }
  describe() {
    return `${this.name} the animal`;
  }
}

class Dog extends Animal {
  constructor(name, breed) {
    super(name);
    this.breed = breed;
  }
  bark() { return "woof"; }
}
```

WHAT IT DESUGARS TO

```
function Animal(name) {
  this.name = name;
}
Animal.prototype.describe = function(){
  return `${this.name} the animal`;
};

function Dog(name, breed) {
  Animal.call(this, name); // super
  this.breed = breed;
}
Dog.prototype =
  Object.create(Animal.prototype);
Dog.prototype.constructor = Dog;
Dog.prototype.bark = function(){ return
"woof"; };
```

Both columns produce the same objects, the same chain, the same behavior. Methods you write in the class body land on `Dog.prototype` / `Animal.prototype` — shared, not per-instance.

Where each kind of member lands

You write	Ends up	Shared across instances?
<code>describe() { ... }</code> (method)	On <code>Animal.prototype</code>	Yes — one copy
<code>this.name = ...</code> or field <code>name = ...</code>	Own property on the instance	No — one per instance
<code>handler = () => {...}</code> (arrow field)	Own property on the instance	No — new function per instance (auto-bound <code>this</code>)
<code>static make() { ... }</code>	On the constructor <code>Animal</code> itself	Called as <code>Animal.make()</code> , not on instances

C# CONTRAST

In C#, methods, fields, and statics are all declared inside the type and the compiler files them in the right table. In JS they land in physically different *places* (prototype object vs. instance vs. constructor), and you can see and change those places at runtime. The class body just chooses the destination for you.

`super.method()` — reusing the parent's version

Inside a method, `super.x()` means "find `x` starting from my parent's prototype, but keep `this` as the current instance." Use it to extend behavior instead of duplicating it:

```
class Dog extends Animal {
  describe() {
    return `${super.describe()}, specifically a dog`; // reuse Animal's string, then add
  }
}
new Dog("Mitzie").describe(); // "Mitzie the animal, specifically a dog"
```

PART 5

`super()` and the derived constructor

This is the part that keeps biting you. Read it slowly once; the model is small, it's just unfamiliar. Everything below is one idea seen from different angles: **in a derived class, the object is born inside the `super()` call, not before it.**

Step by step: what `new Dog("Mitzie")` really does

Given `class Dog extends Animal`:

1. `new Dog("Mitzie")` starts. A hidden value `new.target` is set to `Dog` (it records "what did someone actually write `new` on").
2. Because `Dog` is a *derived* class (it has `extends`), **this does not exist yet**. The `this` slot is empty — reading it throws. The engine deliberately withholds it.
3. The `Dog` constructor body runs until it reaches `super(name)`.
4. `super(name)` runs the `Animal` constructor. Here's the crucial bit: the **base-most constructor** (the one with no `extends` — `Animal`) is where the actual object gets **allocated**. It's created with its `[[Prototype]]` set to `new.target.prototype`.
5. And `new.target` is still `Dog` (it flows down the whole `super` chain). So the object's prototype is `Dog.prototype` — not `Animal.prototype`. `Animal`'s constructor runs its `this.name = name` against that object.
6. `super()` hands that finished object back and binds it to `this` in the `Dog` constructor. **Now `this` exists and is usable.**
7. The rest of the `Dog` constructor runs (`this.breed = breed`), writing onto the same object.

WHY THE ERROR YOU KEEP HITTING HAPPENS

Put `this.breed = breed` before `super(name)` and it throws `ReferenceError: Must call super constructor ... before accessing 'this'`. Reason: at that line the object hasn't been allocated yet — it gets allocated *inside* the `super()` call. You're reaching for an object that doesn't exist. `this` is in the **temporal dead zone (TDZ)** until `super()` returns.

The three facts to nail (these are your exact miss points)

Question	The precise answer
How many objects exist after <code>super()</code> ?	Exactly one. Not a parent object plus a child object. One object that both constructors write onto.
What is <code>this</code> 's prototype, and who set it?	Its <code>[[Prototype]]</code> is <code>Dog.prototype</code> (= <code>new.target.prototype</code>), set when the base-most constructor allocated the object . <code>Animal.prototype</code> is one link further up the chain, not the direct prototype.
What is the state of <code>this</code> on the line before <code>super()</code> returns?	Uninitialized — in the TDZ. Touching it throws <code>ReferenceError</code> . It's not <code>undefined</code> , not an empty object; it's "not there yet."

Field initializers, quickly

Instance fields written as `legs = 4;` in a derived class run *right after* `super()` returns, before the rest of your constructor body. In a base class they run at the very start of the constructor. Either way: fields need the object to exist, so derived fields wait for `super()`. Same rule, one cause.

ONE-SENTENCE VERSION

A derived class doesn't own the object creation — it *delegates* it up the chain via `super()`, and gets its `this` back only once the base constructor has allocated the single shared object with `new.target.prototype`.

Why the old-school version didn't have this TDZ problem

Reread Part 3: there, `this` was usable at the top of `Dog` before `Animal.call(this)`. Why? Because with a plain function constructor, `new` creates the object *up front* (Part 2, step 1) and hands it in. The ES6 `class` design deliberately moved object creation *into* the `super()` call so that `new.target.prototype` and subclassing built-ins work correctly. That design choice is the entire reason the TDZ-before-`super()` rule exists. Interesting side effect: **ES6 derived classes behave less like the old JS, and the "object exists first" behavior actually lives in C#** — which is the perfect segue.

PART 6

JavaScript vs C#, side by side

Your C# instinct is *usually* what generates the wrong JS answer here — because C# genuinely does it in a different order. Seeing the two next to each other stops the instinct from leaking in.

The core difference: *when* the object comes into existence

	C#	JavaScript (derived class)
Allocation	The <i>whole</i> object — memory for all fields of base <i>and</i> derived — is allocated up front, before any constructor runs , and zero-initialized (0 / null / false).	The object is allocated inside the base-most constructor call , which is reached via <code>super()</code> . Nothing exists before that.
Constructor order	Base constructor runs first, then derived — over an object that already physically exists.	Same visible order (parent work happens at <code>super()</code> , then child) — but the object is <i>produced by</i> that parent call, not pre-existing.
this before base runs	Fully valid. The object exists; fields are just at their default values. You literally cannot observe "no object yet."	In the TDZ. Reading <code>this</code> before <code>super()</code> returns throws <code>ReferenceError</code> .
The type / prototype	The object is a <code>Dog</code> from the instant of allocation.	Its <code>[[Prototype]]</code> is <code>Dog.prototype</code> (<code>new.target.prototype</code>) from the instant the base constructor allocates it.

THE CLEAN WAY TO SAY IT IN AN INTERVIEW

"In C#, the runtime allocates the entire object — every base and derived field — and zero-initializes it *before* any constructor body runs, so `this` is valid from the first line and the ordering question can't arise. In JS, a derived class has no object until `super()` allocates it in the base constructor, so `this` is in a temporal dead zone until `super()` returns. It's an allocation-order difference, not a compile-time-vs-runtime difference."

KILL THIS PHRASING

"The difference is C# is compile-time and JS is runtime." That's the answer that's been marked wrong twice. It's not about *when types are checked*; it's about *when the object is allocated* relative to constructor code. Say **allocation order**, every time.

Bonus footgun (worth knowing, shows depth)

Because C# allocates the real `Dog` up front, if the `Animal` (base) constructor calls a `virtual` method that `Dog` overrides, C# dispatches to **Dog's override** — running against a

Dog whose own fields aren't initialized yet. Classic C# trap. In JS, methods are looked up on the same live prototype chain, and the analogous call from within a parent constructor likewise reaches the child's override — same hazard, same root cause (the object is already the derived type). Different languages, same lesson: don't call overridable methods from constructors.

PART 7

`this`, detachment, and `bind`

This isn't about classes exactly, but it's the other place your answers wobble, and it shares a root with the above: `this` is decided by *how a function is called*, not by where it was defined.

The one rule for `this`

`this` is set at the **call site**, by the *form* of the call — not by lexical scope, not by where the method lives:

Call form	<code>this</code> is
<code>obj.method()</code>	<code>obj</code> — the thing left of the dot
<code>const f = obj.method; f()</code>	<code>undefined</code> (strict mode) — the receiver was <i>detached</i>
<code>fn.call(x)</code> / <code>fn.apply(x)</code>	<code>x</code> — set explicitly, for this one call
<code>fn.bind(x)</code> then call it later	always <code>x</code> — permanently pinned
arrow function	inherited from the enclosing scope, ignores the call form entirely

THE PRECISE TRIGGER FOR NEEDING `BIND`

You need `bind` when a **method reference gets detached from its object and invoked later by something else** — e.g. you pass `this.handleClick` to `addEventListener`. The listener calls your function with no dot, so `this` would be lost. `bind` pins it. It is **not** "needed when calling from a scope where `this` is undefined" — `this` isn't about scope at all. It's about receiver detachment at the call site.

```
class Widget {
  constructor() {
    this.handleClick = this.handleClick.bind(this); // needed: detached into
    addEventListener
    // this.refresh = this.refresh.bind(this); // pointless if only ever called as
    widget.refresh()
  }
  handleClick() { console.log(this); }
  refresh() { /* ... */ }
}
el.addEventListener("click", widget.handleClick); // detached – bind saved it
widget.refresh(); // dot form – `this` is widget already, no bind required
```

C# CONTRAST

In C#, converting a method group to a delegate (`Action a = obj.Method;`) *captures the receiver* into the delegate (`Target + Method`). The receiver can't be lost by passing the delegate around. `bind` is you doing that capture by hand — JS doesn't do it for you, which is why the detachment bug exists at all.

What binding in the constructor actually does

Common misconception: "binding makes `this` point at the prototype." It doesn't. `this.m = this.m.bind(this)` creates a *new* bound function and stores it as an **own property on the instance**, which *shadows* the prototype method. `this` still resolves to the instance. The real costs: one extra function allocation per instance per bound method, and `instance.m !== Proto.prototype.m` — which quietly breaks `removeEventListener`, test spies, and later monkey-patching of the prototype.

PART 8

Cheat sheet + your mistakes, corrected

The chain, one more time

```
instance → Sub.prototype → Base.prototype → Object.prototype → null
|           |           |
own data    Sub's methods  Base's methods
(this.x)    (shared)       (shared)
```

Reading a property walks this chain left-to-right until found or null.
super() = "run the parent constructor on THIS one object" (= Parent.call(this))
super.m() = "start method lookup at the parent's prototype, keep this = me"
new.target = the class you actually wrote `new` on (flows down the super chain)

Instant answers

Prompt	Answer
How many objects in an instance of <code>Dog extends Animal</code> ?	One.
<code>this</code> 's prototype right after <code>super()</code> ?	<code>Dog.prototype</code> (= <code>new.target.prototype</code>).
Who allocated the object?	The base-most constructor, when <code>super()</code> reached it.
<code>this</code> before <code>super()</code> returns?	TDZ. Reading it → <code>ReferenceError</code> .
What does <code>super(x)</code> do?	Runs the parent constructor against the same object; produces & binds <code>this</code> .
Methods live where?	On the prototype (shared). Data lives on the instance.
C# vs JS constructor difference?	Allocation order: C# allocates the whole object first; JS allocates inside the base ctor via <code>super()</code> .
When is <code>bind</code> actually needed?	Receiver detachment — method reference invoked later by something else.

Your recurring misses, stated correctly

MISS 1 — "SUPER() CREATES A PARENT INSTANCE THE CHILD IS LAYERED ONTO"

Correct: There is one object. `super()` runs the parent constructor *against that one object* (think `Animal.call(this)`). No second instance is ever created.

MISS 2 — "THIS'S PROTOTYPE IS ANIMAL"

Correct: `this`'s direct `[[Prototype]]` is `Dog.prototype` (`= new.target.prototype`). `Animal.prototype` is one link further up. The base constructor set the prototype, using `new.target` — and `new.target` is `Dog`.

MISS 3 — DIDN'T NAME THE PRE-SUPER() STATE

Correct: Before `super()` returns, `this` is in the **temporal dead zone**; reading it throws `ReferenceError`. Not `undefined`, not `{}`.

MISS 4 — "C# VS JS = COMPILE-TIME VS RUNTIME"

Correct: It's an **allocation-order** difference. C# allocates and zero-initializes the entire object before any constructor body runs; JS allocates the object inside the base-most constructor, reached through `super()`.

IF IT EVER SLIPS AGAIN

Fall back to the mantra: "One object. `super()` is `Parent.call(this)`. The object is born inside that call, with `Dog.prototype`, and I can't touch `this` until it returns." Everything in this guide is a footnote to that sentence.